

Draw It or Lose It

CS 230 Project Software Design Template

Version 1.2

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 5
Recommendations 9

Document Revision History

Version	Date	Author	Comments
1.0	09/15/26	Nick Hardy	Executive summary, Requirements, Constraints, Domain first iteration complete.
1.1	10/11/26	Nick Hardy	Completed operating platform evaluation for Project Two.
1.2	10/22/26	Nick Hardy	Completed system architecture recommendations, including operating platform, storage, memory management, distributed systems, and security.

Executive Summary

The Gaming Room currently offers Draw It or Lose It as an Android application and would like to expand the game into a web-based application that can serve users across multiple platforms. The new application must preserve the existing gameplay while supporting multiple games, teams, and players. Each game will contain one or more teams, and each team will contain multiple players. Game and team names must remain unique, and every game, team, and player must receive a unique identifier.

To meet these requirements, Creative Technology Solutions proposes developing the application for a distributed web environment using an object-oriented design. A single GameService instance will manage the collection of active games through the singleton design pattern, while iterator functionality will search existing collections and prevent duplicate names. A shared Entity base class will contain the identification and naming features common to games, teams, and players, allowing these classes to reuse code through inheritance. This design provides an organized and maintainable foundation for expanding Draw It or Lose It beyond Android while allowing users on different platforms to access the application through a web browser.

Requirements

Business requirements

- Expand Draw It or Lose It beyond its current Android-only version.
- Make the game available across multiple platforms.
- Preserve the existing gameplay.
- Support games containing teams and teams containing players.
- Prevent duplicate game and team names

Technical requirements

- Build the application for a web-based distributed environment.
- Allow each game to contain one or more teams.
- Allow each team to contain multiple players.

- Assign unique identifiers to every game, team, and player.
- Use a single GameService instance to manage active games.
- Use the Singleton pattern to control access to GameService.
- Use iterator logic to search for duplicate names.
- Use an Entity base class for the shared id and name attributes.
- Use inheritance so Game, Team, and Player reuse the common behavior from Entity.

Design Constraints

The primary design constraint is that Draw It or Lose It must operate as a web-based application in a distributed environment. Unlike the current Android-only version, the application must support users on different operating systems and devices. This requires the client side of the application to use platform-independent, browser-compatible technologies instead of relying on features available only on Android. Differences among browsers, screen sizes, and input methods must also be considered so that users receive consistent functionality across desktop and mobile devices.
The distributed environment also requires clients to communicate with a central server over a network. This creates a dependency on reliable connectivity and requires the server to manage multiple users and game sessions at the same time. The application must be designed to handle changes in network performance, temporary disconnections, and an increasing number of users without losing game data or creating inconsistent results. Communication between clients and the server must also be secured to protect user information and prevent unauthorized access.

The client's gameplay requirements further constrain the internal design. Each game must contain one or more teams, and each team must support multiple players. Games, teams, and players require unique identifiers, while game and team names must be checked before new objects are created to prevent duplicates. A single GameService instance must manage the active games so that all parts of the application access consistent data. These constraints require controlled object creation, collection searches, and a clear class structure that can be maintained as the application expands.

System Architecture View

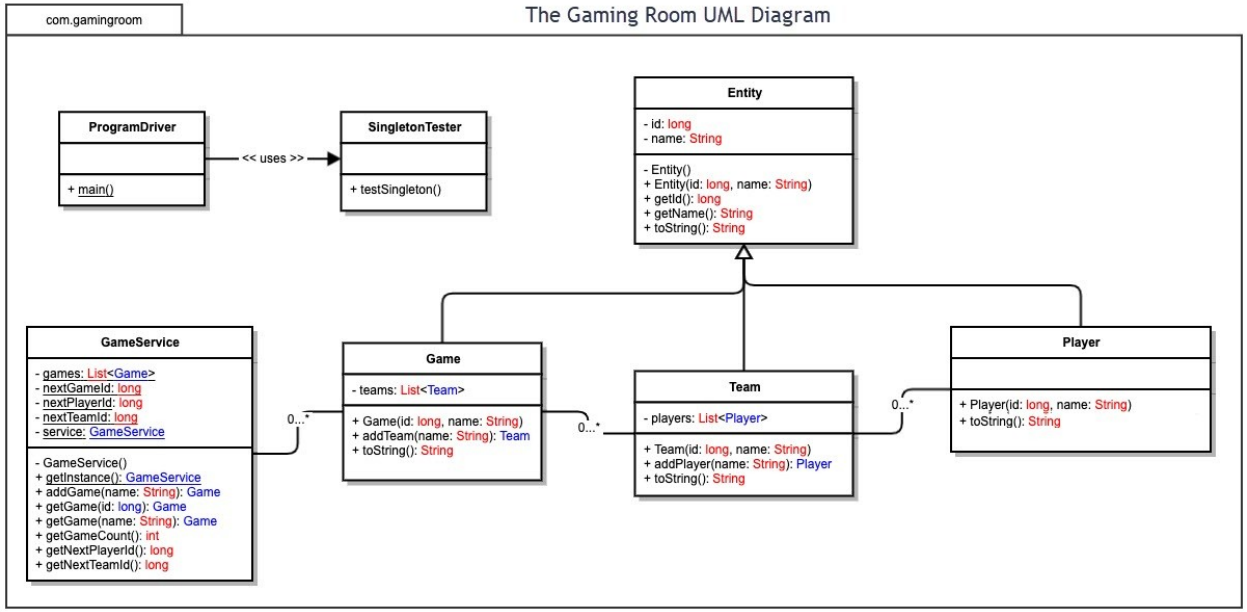
Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The UML class diagram represents the domain model for the Draw It or Lose It application. The GameService class manages the collection of active Game objects and assigns unique identifiers to games, teams, and players. Its private constructor, static service attribute, and public getInstance() method implement

the singleton pattern, ensuring that only one GameService instance manages the application's game data. GameService is associated with zero or more Game objects, allowing the service to begin with an empty collection and manage multiple games as they are created.
The Entity class is the base class for Game, Team, and Player. It contains the id and name attributes, along with the constructors, accessor methods, and toString() behavior shared by these entities. Game, Team, and Player inherit this common state and behavior, which reduces duplicated code and ensures that every object receives an identifier and name. A Game is associated with zero or more Team objects, while each Team is associated with zero or more Player objects. Although these collections may initially be empty, the application can add teams and players as a game is configured. The addTeam() and addPlayer() methods search the existing collections before creating new objects, helping enforce the requirement that names remain unique.

The diagram demonstrates inheritance, encapsulation, and polymorphism. Inheritance allows Game, Team, and Player to reuse the fields and methods defined by Entity. Encapsulation protects the id, name, and collection attributes by declaring them private and providing controlled access through public methods. Polymorphism is demonstrated by the subclasses providing their own versions of the toString() method inherited from Entity. These principles create a maintainable design that supports the required relationships among games, teams, and players while reducing repeated code. ProgramDriver starts and tests the application, while its dependency on SingletonTester allows the program to verify that the same GameService instance is used throughout the application.



Evaluation

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	macOS can host web-based applications using Java and web server software such as Apache. Its Unix-based architecture provides networking, security, and process management capabilities needed to operate a server. However, macOS is primarily designed for desktop computing rather than large-scale server deployments. Apple's hardware and licensing restrictions limit deployment flexibility and can increase infrastructure costs. Although macOS could support The Gaming Room's application, scaling the environment to accommodate thousands of players would be less practical than using a dedicated server operating system. Since the application will be accessible through web browsers, hosting it on macOS would provide no significant advantage to	Linux is well suited for hosting web-based applications in distributed environments. It supports Java, web server software, and networking services, allowing The Gaming Room to host its application and communicate with clients across multiple operating systems. Linux also provides efficient resource management and supports virtualization, containers, and cloud-based deployments that can help the application scale as the number of players increases. Many Linux distributions are available without operating system licensing fees, although enterprise support and hosting services may involve additional costs. Its flexibility, scalability, and lower potential licensing expenses make Linux a strong option for hosting Draw It or Lose It	Windows Server provides a reliable environment for hosting web-based applications through technologies such as Internet Information Services (IIS), Java, and other server frameworks. It includes graphical administration tools, security features, and support for virtualization and distributed applications. These capabilities would allow The Gaming Room to host its application and accommodate increasing numbers of players. However, Windows Server requires commercial licensing, with additional licensing requirements depending on whether users access internal services or a public-facing application. These expenses could increase the overall cost of maintaining the application. Windows Server remains a viable hosting platform, particularly for	Mobile operating systems such as Android and iOS are primarily designed to run client applications rather than host large-scale web services. Although some mobile devices can run limited server software, restrictions involving processing resources, background services, network connectivity, and power management make them unsuitable for hosting an application supporting thousands of players. Mobile operating systems generally do not require separate server licensing fees, but they also lack the deployment flexibility of dedicated server platforms. For The Gaming Room, mobile devices would be better suited as clients communicating with a centralized application server. This approach allows Android and iOS users to participate without requiring the application to be hosted

Development Requirements	Mac	Linux	Windows	Mobile Devices
	users accessing the game from Apple devices.	while supporting thousands of concurrent players.	organizations already using Microsoft infrastructure, but it may be less cost-effective than Linux.	directly on their devices.
Client Side	Supporting macOS clients would require developing a responsive web interface using HTML, CSS, and JavaScript that can run in browsers such as Safari, Chrome, and Firefox. Since the application would operate through a browser, The Gaming Room would not need to develop a separate native macOS application. This would reduce development time and costs by allowing the same front-end code to be shared across operating systems. However, testing would still be necessary to identify differences in browser behavior, rendering, and compatibility. The development team would need experience with front-end web technologies and cross-browser testing. Access to macOS hardware for testing could introduce	Linux clients could access Draw It or Lose It through modern web browsers such as Firefox and Chrome without requiring a separate Linux application. Using HTML, CSS, and JavaScript would allow The Gaming Room to maintain the same web interface across multiple desktop operating systems, reducing duplicated development work. Linux also offers free development and testing tools, which could help limit costs. However, differences among Linux distributions, desktop environments, and browser configurations may require additional compatibility testing. Developers would need experience with responsive web design and browser debugging to ensure the application performs consistently. Overall, Linux would	Windows clients could access Draw It or Lose It through web browsers such as Microsoft Edge, Chrome, and Firefox. The browser-based approach would eliminate the need to develop and maintain a separate Windows desktop application, reducing development time and overall costs. Windows provides a widely supported environment for web development and testing, with tools available for identifying compatibility and performance issues. However, the development team would still need to test the application across different browsers, screen resolutions, and input methods to ensure consistent functionality. Developers familiar with HTML, CSS, JavaScript, and browser testing would be able to support Windows clients without	Supporting Android and iOS clients would require a responsive web interface that adapts to smaller screens, touch controls, and different device capabilities. HTML, CSS, and JavaScript could be used to provide access through mobile browsers such as Chrome and Safari without developing separate native applications. This approach would reduce development costs and allow the same core application to serve desktop and mobile users. However, additional time and testing would be required to address differences in screen sizes, browser behavior, network connectivity, and mobile performance. Developers would need experience with responsive design, mobile browser testing,

Development Requirements	Mac	Linux	Windows	Mobile Devices
	additional costs, but these would generally be lower than maintaining a separate desktop application.	provide a cost-effective client platform while remaining compatible with the centralized web application.	requiring an entirely separate development team.	and touch-based interfaces. Testing on representative Android and iOS devices would also be necessary to ensure a consistent gameplay experience. Since the game uses timed rounds and displays drawings progressively, additional testing would be necessary to ensure that players receive consistent updates despite differences in mobile network performance.
Development Tools	macOS supports development tools such as Eclipse, Visual Studio Code, and IntelliJ IDEA for building Java and web-based applications. Java and frameworks such as Dropwizard can be used to develop the server-side application, while HTML, CSS, and JavaScript can provide the browser-based interface. Many of these tools are available at no cost, although some commercial IDE editions require licensing. Apple's Xcode is also available for testing and	Linux provides a flexible development environment with support for Java, Eclipse, Visual Studio Code, Maven, and frameworks such as Dropwizard. These tools would allow developers to build, test, and deploy the server-side application while using HTML, CSS, and JavaScript for the client interface. Many Linux development tools are free and open source, reducing software licensing expenses. Linux also provides command-line tools, package	Windows supports development tools such as Eclipse, Visual Studio Code, IntelliJ IDEA, and Visual Studio. Java, Maven, and Dropwizard could be used to develop the application's back end, while HTML, CSS, and JavaScript would support the web-based interface. Many tools are available in free editions, although commercial software licenses may increase development costs depending on the tools and organization. Windows also provides debugging and testing	Development for Android and iOS would primarily involve HTML, CSS, and JavaScript to create a responsive browser-based interface. Tools such as Visual Studio Code, browser developer tools, and mobile device emulators could be used to develop and test the application. Android Studio and Apple's Xcode could also assist with platform-specific testing, although native mobile development would not be required. These tools are generally available

Development Requirements	Mac	Linux	Windows	Mobile Devices
	developing native Apple applications, but it would not be required for the proposed browser-based design. Developers would need experience with Java, web technologies, and cross-platform compatibility. Since the application would use a shared codebase, a separate macOS development team would not be necessary, although access to Apple hardware would be beneficial for testing.	managers, and container technologies that can simplify deployment and maintenance. Developers would need experience with Java, web development, Linux administration, and server configuration. Because these technologies are compatible with multiple operating systems, the same development team could maintain the application without needing separate teams for each platform.	tools for browsers such as Edge, Chrome, and Firefox. Developers would need knowledge of Java, web development, and browser compatibility, but the use of shared technologies would reduce the need for a dedicated Windows development team. This approach would help The Gaming Room maintain a consistent application across operating systems.	without software licensing fees, but Apple hardware, physical test devices, and certain commercial testing services could introduce additional costs. Developers would need expertise in responsive design, mobile browser compatibility, and touch-based interactions. A shared web development team could support both platforms, reducing the time and expense associated with maintaining separate Android and iOS applications.

Recommendations

1. **Operating Platform**:
   Linux is the recommended server operating system for hosting Draw It or Lose It. It provides a stable, flexible, and cost-effective environment for deploying web-based applications. Linux supports Java, web server technologies, and networking services, making it compatible with the application's existing design. Distributions such as Ubuntu Server also provide tools for managing system resources, security, and application services.

   Linux is well suited for distributed environments because it supports virtualization, containerization, and cloud-based deployment. These capabilities would allow The Gaming Room to expand its server infrastructure as the number of players increases. Many Linux distributions are available without operating system licensing fees, helping reduce costs compared to commercial server platforms.

   By hosting the application on Linux and providing a browser-based interface, The Gaming Room can support players using Windows, macOS, Linux, Android, and iOS without developing separate applications for each platform. This approach provides a scalable and maintainable foundation for expanding Draw It or Lose It.

2. **Operating Systems Architectures**:
   Linux uses a monolithic kernel architecture with modular components that allow additional functionality to be loaded as needed. The kernel manages essential system resources, including processes, memory, file systems, hardware devices, and network communication. Linux separates kernel space from user space, allowing applications to operate without directly accessing protected system resources. Applications request operating system services through system calls, providing controlled access to hardware and other resources.

   Draw It or Lose It would run as a Java application in user space, using the Java Virtual Machine (JVM) to execute its code. The JVM interacts with the Linux operating system to manage threads, allocate memory, and perform network operations. Linux's process scheduling and multitasking capabilities would allow the server to handle multiple player requests concurrently while maintaining system stability.

   This architecture provides The Gaming Room with a flexible and secure environment for hosting its application. Linux can support multiple application instances and distribute workloads across available resources, making it suitable for a growing web-based gaming platform.

3. **Storage Management**:
   The ext4 file system is recommended for managing storage on The Gaming Room's Linux server. Ext4 supports large files and storage volumes, file permissions, and journaling to help maintain file system consistency following unexpected shutdowns. It is widely supported by Linux distributions and provides a reliable foundation for storing application files, logs, and other server resources.

A relational database such as PostgreSQL would be appropriate for managing game data, player information, team records, and unique identifiers. The database would provide structured storage, support concurrent access, and help maintain data consistency as multiple users interact with the application. The game's library of stock drawings could be stored as files or in object storage, with the database maintaining references to those images.

Combining Linux storage management with a centralized database would allow The Gaming Room to organize its data efficiently and support multiple application instances. Regular backups and appropriate access permissions would also be necessary to protect stored information and support recovery from system failures.

4. **Memory Management**:
   Linux uses virtual memory and paging to manage system memory efficiently while allowing multiple processes to operate simultaneously. Virtual memory provides each process with its own address space, helping isolate applications and prevent unauthorized access to another process's memory. Paging divides virtual memory into fixed-size pages that can be mapped to physical memory as needed. Linux can also reclaim memory and use swap space when necessary, although excessive swapping can reduce performance.

   Draw It or Lose It would run within the Java Virtual Machine (JVM), which manages memory allocated to Java objects through automatic garbage collection. This allows memory occupied by objects that are no longer reachable to be reclaimed. Linux would manage the underlying memory resources while the JVM handles application-level memory allocation and cleanup.

   These techniques would allow The Gaming Room to manage multiple game sessions and player requests efficiently. Monitoring memory usage and allocating sufficient RAM would be important as the number of concurrent players increases, helping maintain consistent performance and prevent resource exhaustion.

5. **Distributed Systems and Networks**:
   Draw It or Lose It should use a distributed client-server architecture that allows players on different operating systems to communicate with a centralized application service. Clients running Windows, macOS, Linux, Android, or iOS would access the game through web browsers, while Linux servers would manage game sessions, player information, and application logic. Communication between clients and servers would occur over HTTPS using REST APIs, with WebSockets providing real-time updates for drawings, guesses, and game timers.

   To support thousands of concurrent players, The Gaming Room could distribute incoming requests across multiple application servers using a load balancer. A shared database would maintain consistent game and player information across server instances. This would also require coordination of active game sessions and unique identifiers, since the GameService Singleton only manages data within an individual application process.

   The distributed environment would depend on reliable network connectivity and communication between clients, servers, and shared storage. Network interruptions or server failures could disrupt gameplay or cause temporary loss of access. Connection monitoring,

automatic reconnection, redundant servers, and regular data backups would help reduce disruptions and support recovery. These measures would provide a scalable and reliable environment while maintaining consistent gameplay across different platforms.

6. **Security**:
   Security for Draw It or Lose It should combine Linux operating system protections with application-level security measures. Linux provides user and group permissions, file access controls, process isolation, and firewall capabilities to help prevent unauthorized access to server resources. The Gaming Room should also maintain security updates, restrict administrative privileges, and configure services to operate with only the permissions necessary to perform their functions.

   The application should require user authentication to verify player identities and authorization to control access to accounts and game resources. Role-based access control could distinguish between regular players and administrators. User passwords should be stored using secure, salted password hashing rather than plaintext, while sensitive application data should be protected through appropriate database permissions and encryption where necessary.

   Communication between clients and servers should use HTTPS with TLS encryption to protect information transmitted across networks. WebSocket connections should also use secure WSS connections. These measures, combined with secure session management, server monitoring, and regular backups, would help The Gaming Room protect user information and maintain a secure gaming environment across multiple platforms.